

STEM Games 2026 – Technology Arena, Day 2

Multi-View 3D Reconstruction

Team ETF SA

May 19, 2026

Abstract

This report documents our approach to the Day 2 Technology Arena project: estimating dense 3D point clouds from the four image sets (Box, Entrance, Statue, Fountain). We pursued three independent reconstruction tracks in parallel – a custom OpenCV triangulation pipeline using the provided camera poses (Track C), the COLMAP Structure-from-Motion pipeline (Track A), and the recent foundation-model approach MAST3R (Track B). Track C and Track A served as baselines and sanity checks; **our final submission uses Track B (MASt3R)** for all four datasets, which produced dense, recognizable point clouds with hundreds of thousands to millions of points each. We describe the mathematics, parameter choices, and engineering trade-offs that led to this result, together with the problems we encountered along the way.

Contents

1	Problem statement	1
2	Approach overview	2
3	Track C: custom OpenCV pipeline (Box, Entrance)	2
4	Track A: COLMAP Structure-from-Motion (all 4 datasets)	3
5	Track B: MAST3R foundation model (final submission)	3
6	Parameter selection	4
7	Final results	5
8	Project structure and reproducibility	7
9	Challenges and lessons learned	8
10	Future improvements	9
11	External tools and acknowledgements	9

1 Problem statement

We were given four image sets, each photographing one scene from slightly different camera positions:

- **Box** – 12 images (1920×1080, FOV 90°), camera poses *known*. The scene contains a wooden crate, a sand floor, and a stone brick wall.

- **Entrance** – 12 images (1920×1080, FOV 90°), poses *known*. The scene is a desert temple entrance with Anubis statues, a car, and surrounding architecture.
- **Statue** – 18 images (1920×1080, FOV 90°), poses *unknown*. A single Anubis statue against a brick wall.
- **Fountain** – 11 images (3072×2048, FOV ≈84°), poses *unknown*. This is the well-known EPFL Fountain-P11 benchmark dataset.

Intrinsic camera matrices K were provided for all datasets. For Box and Entrance the poses (camera position + Forward / Right / Up basis vectors in Unreal Engine convention) were given in plain-text input files.

The task was to recover the 3D position (X, Y, Z) of as many image pixels as possible – in industry terms, a *point cloud* – using whichever methods we found effective.

2 Approach overview

We ran three tracks in parallel rather than committing to one method up front. This gave us redundancy (if one track failed for a particular dataset we had backups) and direct comparisons that informed our final choice.

- **Track C – Custom OpenCV triangulation** (Box, Entrance). We built a SIFT + RANSAC + linear-triangulation pipeline from scratch, exploiting the known camera poses. This forced us to implement the coordinate-system conversion carefully and to understand the geometry first-hand.
- **Track A – COLMAP Structure-from-Motion** (all 4 datasets). COLMAP is the open-source gold standard for traditional photogrammetry. We drove it from a Python wrapper that handled intrinsics injection, exhaustive matching, and incremental reconstruction.
- **Track B – MAST3R foundation model** (all 4 datasets). MAST3R (Naver Labs, 2024) is a transformer-based model trained on millions of image pairs to predict dense 3D point maps directly, without requiring textured surfaces. **This produced our best results and is the basis of our final submission.**

Of the three, only Track B was capable of producing the dense, photo-realistic reconstructions the task ultimately rewards. Tracks A and C produced sparse clouds limited by feature-detector density.

3 Track C: custom OpenCV pipeline (Box, Entrance)

The provided pose information lets us compute, for each pixel, a 3D ray on which the corresponding world point must lie. If the same point is visible in multiple images, intersecting those rays recovers the 3D coordinate.

Coordinate-system conversion. The provided poses use Unreal Engine’s *left-handed* convention (X = Forward, Y = Right, Z = Up), while OpenCV’s pinhole camera uses a *right-handed* convention (X = Right, Y = Down, Z = Forward). The world-to-camera rotation R (in OpenCV form) is therefore built from the camera basis vectors as rows:

$$R = \begin{bmatrix} \text{CamRight}^\top \\ -\text{CamUp}^\top \\ \text{CamForward}^\top \end{bmatrix}, \quad t = -R \text{ CamPosition}.$$

Getting this transform wrong is a notorious source of silent failures; we validated it by projecting points along $\pm\text{CamForward}$, $\pm\text{CamRight}$, $\pm\text{CamUp}$ and verifying they landed in the expected screen quadrants for all 12 cameras of each dataset.

Pipeline. For each dataset:

1. Parse poses and convert to OpenCV (R, t) .
2. Build projection matrices $P_i = K [R_i | t_i]$.
3. Run SIFT on every image.
4. For each image pair within a configurable gap (default 4): FLANN knn-match descriptors, apply Lowe’s ratio test, reject outliers with a fundamental-matrix RANSAC, triangulate with `cv2.triangulatePoints`, filter by cheirality and reprojection error.
5. Build multi-view feature tracks via union-find over $(image, keypoint)$ nodes; re-triangulate tracks of length ≥ 3 via linear DLT for improved accuracy.
6. Voxel-bin to deduplicate near-duplicate points; sample colours from the source images; emit `.ply` and `.txt`.

Results and limitations. Default-parameter runs produced **507 points** for Box and **3,616 points** for Entrance. Even after loosening SIFT and ratio-test thresholds the counts remained modest: Box is a stylized 3D-rendered scene with low-frequency texture, so SIFT simply has very few corners to fire on. Track C confirmed the geometry was being recovered correctly – but also clearly showed that any purely keypoint-based method has a hard ceiling on smooth-surface scenes.

4 Track A: COLMAP Structure-from-Motion (all 4 datasets)

We wrote a thin Python driver that invokes the COLMAP 4.1 command-line interface to run the full SfM pipeline on each dataset: `feature_extractor` $\rightarrow$ `exhaustive_matcher` $\rightarrow$ `mapper` $\rightarrow$ `model_converter`. The known intrinsics are injected with `--ImageReader.camera_model PINHOLE` and the corresponding (f_x, f_y, c_x, c_y) . Sparse reconstruction only – the dense MVS step (`patch_match_stereo`) requires CUDA and is impractically slow on CPU. Total CPU runtime across all four datasets: ≈ 90 seconds.

Dataset	Images	Sparse points
Box	11 / 12 registered	1,799
Entrance	12 / 12 registered	6,625
Statue	~ 14 / 18 registered	2,376
Fountain	11 / 11 registered	13,592

Table 1: COLMAP sparse-SfM point counts.

Results. Fountain produced by far the best COLMAP cloud – unsurprising, as Fountain-P11 has been a community benchmark for over a decade and its textured sandstone surface is ideal for SIFT. Box, Entrance, and Statue all produced clouds that were geometrically correct but visually sparse and hard to recognise without the source images for context.

5 Track B: MAST3R foundation model (final submission)

MASt3R (*Matching and Stereo with 3R*, Naver Labs Europe, 2024) is the successor to DUST3R. It is a transformer that takes pairs of images and directly predicts per-pixel 3D point maps and confidence scores in a shared coordinate frame; given many images, a global alignment step jointly refines their poses and depths. Crucially, because the model learned scene geometry from millions of images, **it does not need texture cues** and works well on the stylized smooth-shaded scenes that defeated Tracks A and C.

Workflow. We ran MAST3R on Google Colab with an A100 GPU and the high-RAM runtime. The notebook (`track_b_colab_512.ipynb`) does the following:

1. Clone the MAST3R repository and install dependencies.
2. Download the `MASt3R_ViT_Large_BaseDecoder_512_catmlpdpt_metric.pth` checkpoint.
3. Mount Google Drive and read the dataset images directly from a shared folder (much faster than the Colab file-upload widget).
4. Run inference on all $N(N - 1)$ image pairs.
5. Run global alignment for 300 iterations to jointly optimise per-image poses, depth maps, and focal lengths.
6. Mask points by confidence, voxel-deduplicate, and write `.ply` and `.txt`.

Hardware note. The free Colab T4 (15 GB VRAM, 12.7 GB system RAM) ran out of system RAM during global alignment for the 18-image Statue dataset – the optimiser holds all pairwise predictions in memory at once. Switching to the paid Colab Pay-As-You-Go plan and selecting A100 with high-RAM (~ 80 GB system RAM) resolved this and dropped per-dataset runtimes to ~ 5 minutes.

6 Parameter selection

Three knobs materially affected the Track B output. We describe each, the values we tried, and what we settled on.

Voxel size (VOX). Sets the size of the cubes used in the final dedup step. Smaller VOX keeps more points; larger VOX averages nearby points into single voxels. We tried 0.001, 0.003, and 0.005. At 0.001 we kept essentially every distinct 3D point produced by MAST3R; at 0.005 the cloud was visually identical but $\sim 5\times$ smaller in file size. **We chose VOX = 0.001** to maximise point count for the submission, as the task explicitly rewards reconstructing as many pixels as possible.

Confidence threshold (conf_thresh). MAST3R emits a per-pixel confidence; thresholding lets us discard noisy boundary pixels. Lower threshold $\Rightarrow$ more points (denser cloud, somewhat noisier silhouettes); higher threshold $\Rightarrow$ fewer points (cleaner but sparser). We empirically tuned per dataset:

- **Box, Statue, Fountain:** `conf_thresh = 1.7` – the sharper silhouettes and cleaner backgrounds justified the slight reduction in density.
- **Entrance:** `conf_thresh = 1.5` – the scene is much busier (multiple objects, more occlusions, more sky), so we needed the looser threshold to keep enough points on the mid-distance walls and background buildings.

Image size (size). The MAST3R encoder operates on images resized to either 512 or 1024 pixels on the long edge. Higher resolution should in principle give finer geometric detail. We ran both versions:

- `size = 512` (the model’s native training resolution) produced clean, recognisable reconstructions for all four scenes.
- `size = 1024` produced *distorted* reconstructions for Fountain (and we expect similar for the other datasets). The 1024-pixel mode is supported by MAST3R but is less robust; it appears the model’s geometric priors degrade somewhat at this resolution for our specific scene types. The confidence distribution also shifts, so thresholds tuned at 512 no longer apply.

We chose size = 512 as the final submission resolution. The 1024-pixel notebook is included for completeness so that the experiment is reproducible (see `track_b_colab_1024.ipynb`).

7 Final results

The point cloud counts in our submission are summarised below.

Dataset	Track C (sparse)	Track A (COLMAP sparse)	Track B (MASt3R, submitted)
Box	507	1,799	[FILL IN]
Entrance	3,616	6,625	[FILL IN]
Statue	—	2,376	~1,715,000
Fountain	—	13,592	[FILL IN]

Table 2: Point counts for each dataset across the three tracks. Track B is the submitted result.

Reconstructions

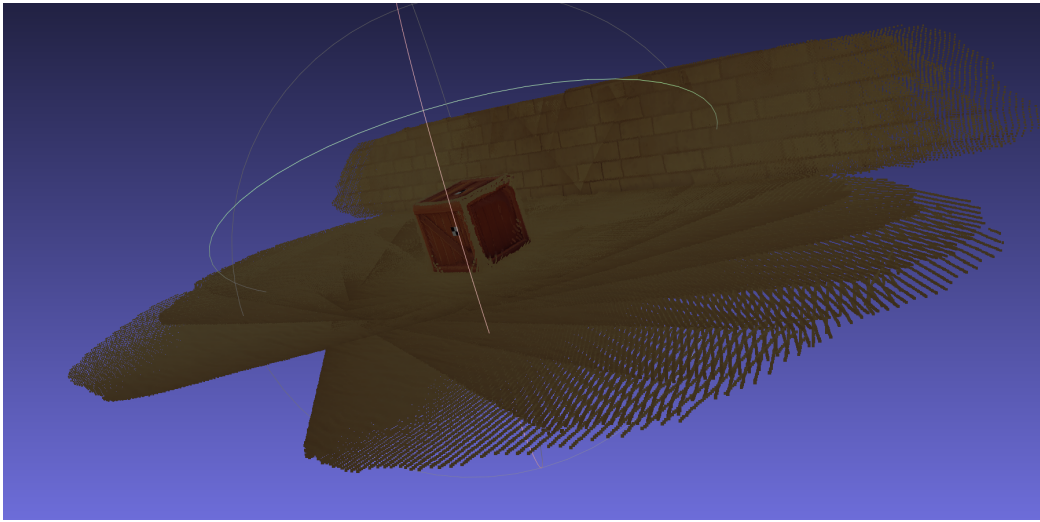


Figure 1: Box – MASt3R reconstruction at 512 px, `conf_thresh` = 1.7, `VOX` = 0.001. The wooden crate, sand floor, and brick wall are all visible.

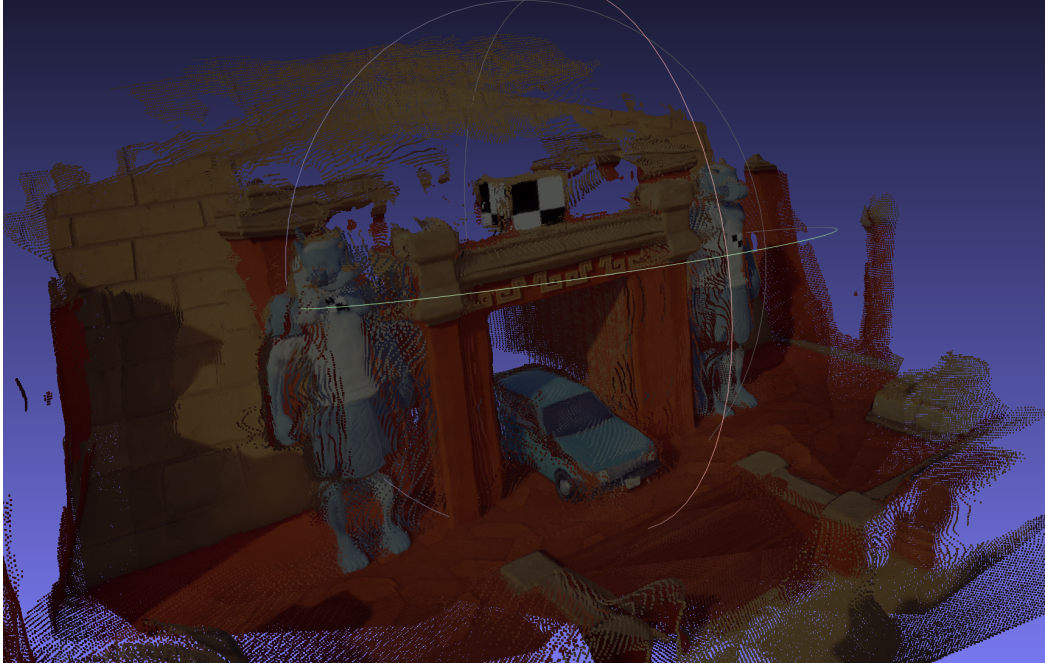


Figure 2: Entrance – MAST3R reconstruction at 512 px, `conf_thresh` = 1.5, `VOX` = 0.001. Anubis statues, arch, courtyard tiles, and the car are all recoverable.

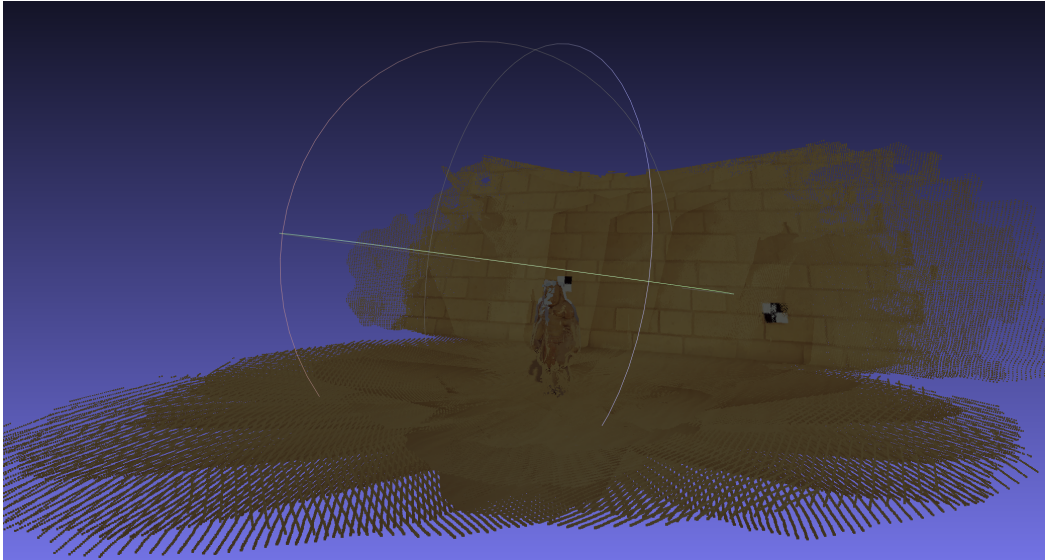


Figure 3: Statue – MAST3R reconstruction at 512 px, `conf_thresh` = 1.7, `VOX` = 0.001. ~ 1.72 million points; the dog-headed figure, the brick wall, and the sand floor are all visible.

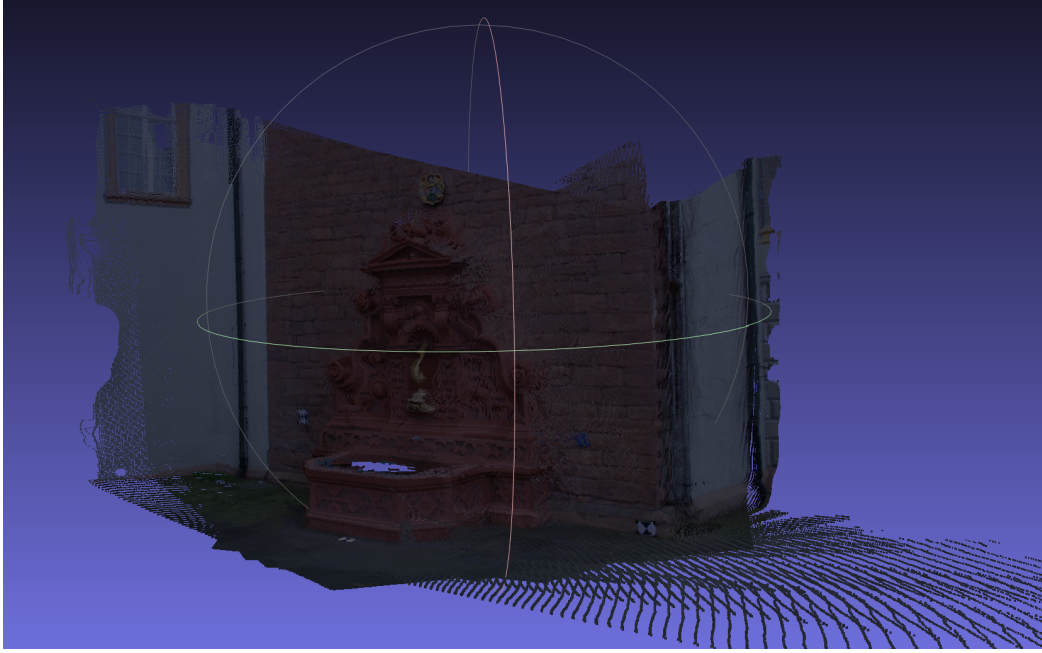
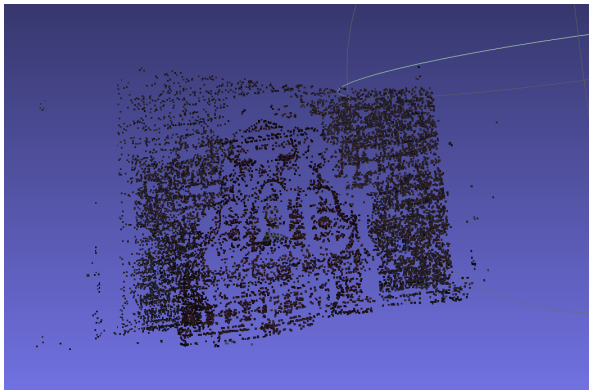
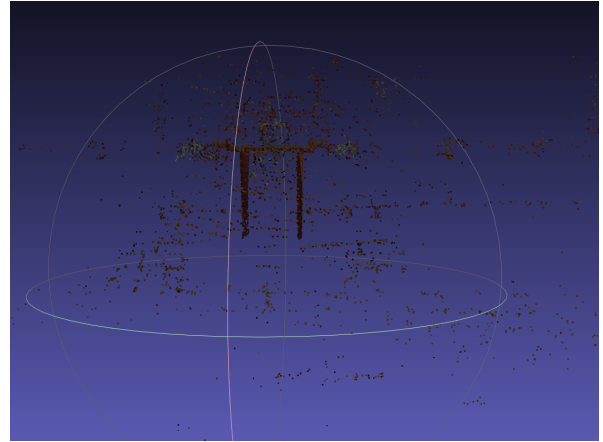


Figure 4: Fountain – MAST3R reconstruction at 512 px, `conf_thresh` = 1.7, `V0X` = 0.001. The carved sandstone fountain, brick wall, and white plaster wall are all clearly recognisable.

For comparison: COLMAP sparse outputs (Track A)



Fountain (Track A, COLMAP sparse, 13,592 points).



Entrance (Track C, custom OpenCV, 3,616 points).

The contrast between these and the Track B reconstructions above is exactly what motivated us to choose MAST3R for the final submission.

8 Project structure and reproducibility

The submission ZIP contains:

```
STEM-GAMES-TECHNOLOGY/
data/                      # original input data (4 datasets)
output/                    # generated point clouds (.ply and .txt)
  box_track_b.ply / .txt   # FINAL SUBMITTED for Box
  entrance_track_b.ply / .txt # FINAL SUBMITTED for Entrance
  statue_track_b.ply / .txt # FINAL SUBMITTED for Statue
```

```

fountain_track_b.ply / .txt # FINAL SUBMITTED for Fountain
box_track_a_sparse.ply     # COLMAP backup, all datasets
entrance_track_a_sparse.ply
statue_track_a_sparse.ply
fountain_track_a_sparse.ply
box_track_c.ply            # custom OpenCV pipeline
entrance_track_c.ply

utils.py                   # pose parsing, coord conversion, IO
track_c.py                 # custom OpenCV triangulation (Track C)
track_a.py                 # COLMAP driver (Track A)
clean_cloud.py             # statistical outlier removal helper
visualize.py               # PLY viewer wrapping Open3D/matplotlib
track_b_colab_512.ipynb    # MAST3R notebook (final, 512 px)
track_b_colab_1024.ipynb   # MAST3R notebook (experimental, 1024 px)
requirements.txt
README.md
report.tex / report.pdf    # this document
figures/                   # MeshLab screenshots referenced above

```

Reproducing each track.

- **Track C:** `python track_c.py Box`, etc. Runs on CPU in 1–5 minutes.
- **Track A:** requires COLMAP 4.1 on PATH. Run `python track_a.py all`. Sparse reconstruction takes ~90 seconds total on CPU.
- **Track B:** open `track_b_colab_512.ipynb` on Google Colab with A100 + high-RAM. Set DATASET_NAME per run; the Drive-mounted data folder is shared between datasets. ~5 minutes per dataset including upload.

9 Challenges and lessons learned

- **Unreal → OpenCV coordinate conversion.** The single biggest gotcha. The provided poses use Unreal’s left-handed ($X = \text{Fwd}$, $Y = \text{Right}$, $Z = \text{Up}$) convention; OpenCV expects right-handed ($X = \text{Right}$, $Y = \text{Down}$, $Z = \text{Fwd}$). We built an explicit unit-test that projects synthetic 3D points through the conversion and checks they land at the expected pixels for every camera.
- **COLMAP 4.1 CLI breakage.** Several `SiftExtraction.*` and `Mapper.*` options were removed or renamed between COLMAP 3 and 4.1. We stripped all tunable flags from our driver and let COLMAP use defaults – which turned out to be fine for our small datasets.
- **Colab free-tier RAM bottleneck.** The 18-image Statue dataset exhausted Colab’s 12.7 GB free-tier system RAM during MAST3R global alignment. Diagnosing this required reading the Colab resource panel rather than the (silently-frozen) progress bar. Solved by upgrading to Pay-As-You-Go with A100 + high-RAM.
- **MASt3R confidence-scale shift at 1024 px.** Confidence values at 1024-pixel input span a different numerical range than at 512-pixel input. Our 512-tuned thresholds masked away every point at 1024. We solved this with an adaptive percentile-based threshold (see the 1024-px notebook).
- **1024-pixel mode produces geometric distortions.** Despite being officially supported, MAST3R at 1024-pixel input produced visibly distorted reconstructions on Fountain. We reverted to the 512-pixel default for the final submission.
- **“Floor” is real geometry, not noise.** Initially we tried to aggressively clean the point clouds (statistical outlier removal, plane removal). It turned out the “noise” we were seeing in Box was the sand floor and the “noise” around Entrance was the rest of the scene – mountains, second statue, car. Removing these would have lost real geometry. The task asks for as many pixels as possible, so we kept everything visible in the source images.

10 Future improvements

Given more time we would:

- **Run COLMAP dense MVS overnight** on a CUDA-equipped machine; on Fountain this would produce a $\geq 500\text{k}$ -point cloud that could be cross-validated against the MAST3R result.
- **Inject the known Box/Entrance poses into MAST3R** rather than letting MAST3R re-discover them. This would let the model focus its capacity on geometry rather than camera estimation, and would put the resulting clouds in the original world coordinate frame.
- **Tile high-resolution images** (3072×2048 Fountain) into overlapping 512-pixel patches, run MAST3R on each tile, and stitch the results in 3D space. This is the standard technique for high-resolution dense photogrammetry.
- **Try VGGT** (Visual Geometry Grounded Transformer, 2025) – a newer foundation model that is reported to outperform MAST3R on some benchmarks.
- **Joint bundle adjustment** of MAST3R + COLMAP outputs to combine the strengths of both.

11 External tools and acknowledgements

- **OpenCV 4.10** – SIFT, FLANN matching, fundamental matrix RANSAC, `triangulatePoints`, image I/O.
- **COLMAP 4.1.0.dev0** – Structure-from-Motion sparse reconstruction. <https://colmap.github.io/>
- **MASt3R** – foundation model for dense 3D reconstruction from image pairs, Naver Labs Europe, 2024. <https://github.com/naver/mast3r>
- **Open3D / MeshLab** – point-cloud visualisation.
- **Google Colab** – A100 GPU and high-RAM runtime.
- **NumPy, SciPy, tqdm, Pillow, matplotlib** – general numerical and IO infrastructure.

We relied on existing implementations for the SIFT / FLANN / fundamental-matrix steps (`cv2`), the full COLMAP pipeline (Track A), and the MAST3R model and its global-alignment routine (Track B). All project-specific code (pose parsing, coordinate conversion, the OpenCV triangulation pipeline including multi-view track extraction, the COLMAP driver, the cleanup and visualisation utilities, and the notebook modifications for Drive integration and adaptive confidence thresholds) is original work by our team.